No-arbitrage for lift lending

The shogting cycle from pallet math

Subtensor lending (PR #3268)
Working note — review response

2026-10-29

Abstract

We analyse the **shogting cycle** (open short → stake/pump → close → unstake) from implemented pallet math. The lift fraction is φ (not a fee); the swap fee is a separate ε. The base case uses **fresh** Finney SN29 reserves and weights at the lending-clone fork block, where $w_q = \frac{1}{2}$. Contaminated clone runs that started from post-drain $w_q \approx 0.47$ are appendix-only. We give closed-form attacker ROI for the unfixed (weight-updating) and fixed (keep-weights) readd, plot ROI against $h = \text{budget}/y$ on a log axis, and state how the derivation relates to μ's September 2026 factor F.

Contents

1 Notation **2**

2 Balancer and lift primitives **2**
2.1 Swaps 2
2.2 Lift and force-sell 2
2.3 Readd (the two designs under test) 3

3 Fresh-fork SN29 base case **3**

4 Shogting cycle: sizing and closed forms **3**
4.1 Sizing from h and lending params 3
4.2 Attacker ROI 3
4.3 Fee-free equal-weight closed forms 4

5 ROI plot **4**

6 Relation to μ's September 2026 derivation **4**

7 Root fix **6**

8 K-neutrality burn **6**

9 Pass/fail (keep-weights), search, slippage **6**

A Contaminated clone runs (not the base case) **7**

1 Notation

Definition 1 (Shogting cycle)**.** The **shogting cycle** is the single-wallet TAO-pure sequence

$$\textbf{open short} \;\to\; \textbf{stake / pump} \;\to\; \textbf{close} \;\to\; \textbf{unstake}.$$

The attacker starts with TAO budget $V = h\,y$ where $h = \text{budget}/\texttt{tao_in}$ and $y = \texttt{SubnetTAO}$. All residual alpha is unwound; the terminal book is pure TAO. This is the only named cycle in this note.

All other symbols are from the live pallet / swap code.

Symbol	Unit	Meaning
x	α	`SubnetAlphaIn`
y	τ	`SubnetTAO`
w_b, w_q	—	Balancer base / quote weights; $w_b + w_q = 1$
L	—	$x^{w_b} y^{w_q}$
p	τ/α	Spot $(w_b/w_q)\,(y/x)$
φ	—	Lift fraction P/x (short)
P	α	Short principal
λ	—	Short LTV (default 0.25)
C	τ	Collateral $= \lambda \cdot$ (debt mark)
S	τ	Pump (`AddStake`)
B	τ	TAO spent to buy repay principal
E	τ	Escrow proceeds from force-sell
ε	—	Swap fee on input (base 0; prod. 33/65535)
h	—	$V/y = \text{budget}/\texttt{tao_in}$
V, V'	τ	Initial / terminal attacker TAO
ROI	—	$V'/V - 1$ (attacker % gain $= 100 \cdot$ ROI)

Remark 1 (Report field `w1`). Clone reports print `w1` with $L \approx x^{1-w_1} y^{w_1}$. That `w1` is our w_q. We write w_q only.

2 Balancer and lift primitives

2.1 Swaps

Fee ε on input, weights fixed mid-swap:

$$\Delta y = y\Big(1 - \Big(\frac{x}{x + \Delta x(1-\varepsilon)}\Big)^{w_b/w_q}\Big), \tag{1}$$

$$\Delta x = x\Big(1 - \Big(\frac{y}{y + \Delta y(1-\varepsilon)}\Big)^{w_q/w_b}\Big). \tag{2}$$

2.2 Lift and force-sell

Lift removes $(\varphi x, \varphi y)$ at unchanged spot. Force-sell $P = \varphi x$. At $\varepsilon = 0$,

$$y_s = y\,(1-\varphi)^{1/w_q}, \qquad E = y\big[(1-\varphi) - (1-\varphi)^{1/w_q}\big], \qquad L_s = (1-\varphi)L. \tag{3}$$

At the fresh-fork equal-weight point $w_q = \frac{1}{2}$, $y_s = y(1-\varphi)^2$ and $E = y\,\varphi(1-\varphi)$.

2.3 Readd (the two designs under test)

Custody returns $(\varphi y, \varphi x)$.

- **Unfixed** (pre-fix / current buggy path): credit via `update_weights_for_added_liquidity` at the post-pump mark (x_c, y_c):

$$w'_q = \frac{w_q\, x_c\, (y_c + \varphi y)}{w_b\, y_c\, (x_c + \varphi x) + w_q\, x_c\, (y_c + \varphi y)}, \qquad w'_b = 1 - w'_q. \tag{4}$$

- **Fixed** (keep-weights; shipped root fix): credit the same deltas with (w_b, w_q) **untouched**.

Does w stay at $\frac{1}{2}$? On the fresh-fork book $w_q = \frac{1}{2}$: under the **fixed** readd, w_q remains $\frac{1}{2}$ through the whole shogting cycle (swaps never reweight; keep-weights readd does not either). Under the **unfixed** readd, w_q moves as soon as the post-pump mark differs from the open mark — typically $w_q < \frac{1}{2}$ after a pump-then-readd.

3 Fresh-fork SN29 base case

Lending-clone documentation: forked from Finney block **#9,378,125**. Archive query of that block:

`SubnetTAO` y	$7897.952\,\tau$
`SubnetAlphaIn` x	$3\,153\,658.330\,\alpha$
`SwapBalancer` w_q	$0.49999999858 \approx \frac{1}{2}$

So the honest base case is **equal weights**, not $w_q \approx 0.47$. The 0.47 / 0.45 figures in the clone a13/a15 reports are **contaminated**: those attacks were run sequentially on one chain (a12 drained first, then a13 started at printed $w_1 = 0.4717$, then a15 at 0.4507). They are kept only as Appendix A.

Script: `scripts/no_arbitrage_shogting.py`.

4 Shogting cycle: sizing and closed forms

4.1 Sizing from h and lending params

Default $\lambda = 0.25$, deep-pool $\varphi_{\max} = \mathrm{Cap}_{\mathrm{short}}/y = 0.165$. Debt mark at equal weight equals φy, so $C = \lambda \varphi y$. Unconstrained we set the pump equal to the lift TAO, $S = \varphi y$, hence

$$\varphi(h) = \min\big(\varphi_{\max},\, h/(1+\lambda)\big). \tag{5}$$

Capacity then binds for $h \geq \varphi_{\max}(1+\lambda) = 0.20625$; extra budget goes to the pump/repay split below.

Wallet-close solvency: after posting C, the remainder $V - C$ splits as $S + B = V - C$ so the attacker can buy repay principal after the pump (B endogenous). Close burns P, refunds $E + C$, readds lift legs, unstakes pump alpha α_p.

4.2 Attacker ROI

$$V' = V - S - B + E + \Delta y_{\mathrm{unstake}}, \qquad \mathrm{ROI}(h) = \frac{V'}{V} - 1, \qquad \%\ \mathrm{gain} = 100 \cdot \mathrm{ROI}(h). \tag{6}$$

4.3 Fee-free equal-weight closed forms

Take $\varepsilon = 0$, $w_b = w_q = \frac{1}{2}$, $\varphi = \varphi(h)$ as in (5), and S, B the solvency split of $V - C$. Write $y_s = y(1-\varphi)^2$, $E = y\varphi(1-\varphi)$, $y_p = y_s + S$, $x_p = x\sqrt{y_s/y_p}$, $\alpha_p = x - x_p$, and

$$B = y_p \left(\frac{1}{\left(1 - \varphi\sqrt{y_p/y_s}\right)^2} - 1 \right) \tag{7}$$

(exact cost to buy $P = \varphi x$). Post-buy state $(x_c, y_c) = (x_p\sqrt{y_p/(y_p+B)},\ y_p + B)$.

Fixed (keep-weights). $x_r = x_c + \varphi x$, $y_r = y_c + \varphi y$, weights stay $(\frac{1}{2}, \frac{1}{2})$. Unstake $\Delta y^{\text{fix}} = y_r \cdot \alpha_p/(x_r + \alpha_p)$. Then

$$\boxed{\text{ROI}_{\text{fix}}(h) = \frac{E - S - B + \Delta y^{\text{fix}}}{V}.} \tag{8}$$

Here w_q remains $\frac{1}{2}$.

Unfixed (weight-updating readd). Apply (4) at (x_c, y_c) to get (w_b', w_q') and $(x_r, y_r) = (x_c + \varphi x,\ y_c + \varphi y)$. Unstake at the new weights $\Delta y^{\text{unfix}} = y_r(1 - (x_r/(x_r + \alpha_p))^{w_b'/w_q'})$. Then

$$\boxed{\text{ROI}_{\text{unfix}}(h) = \frac{E - S - B + \Delta y^{\text{unfix}}}{V}.} \tag{9}$$

Here w_q does **not** stay at $\frac{1}{2}$.

Production fee. Replace every swap by (1)–(2) at $\varepsilon = 33/65535$; lift / readd / (4) unchanged. No closed elementary simplification; the same ROI identity (6) is evaluated numerically (script above).

Theorem 1 (Fresh-fork shogting drain)**.** *On the fork-state SN29 book ($w_q = \frac{1}{2}$), the fee-free shogting cycle has* $\text{ROI}_{\text{unfix}}(h) > 0$ *for large h (weight ratchet), while* $\text{ROI}_{\text{fix}}(h) \le 0$*. At $h = 0.5$: unfixed $+3.10\%$, fixed -2.94%; fixed leaves $w_q = \frac{1}{2}$, unfixed moves $w_q \to 0.473$.*

5 ROI plot

Figure 1: single-wallet shogting cycle on fresh-fork SN29, %ROI vs $h \in [0.01, 10]$ (log), unfixed vs fixed, at $\varepsilon = 0$ and $\varepsilon = 33/65535$.

6 Relation to μ's September 2026 derivation

μ wrote a fee-free pool factor $F(f_h, \varphi, w_2)$ for an earlier shorting-cycle proposal and took attacker profit when $F < 1$ (pool-cost convention). We do **not** assume that formula. We derive the shogting cycle from pallet lift / force-sell / readd, then map:

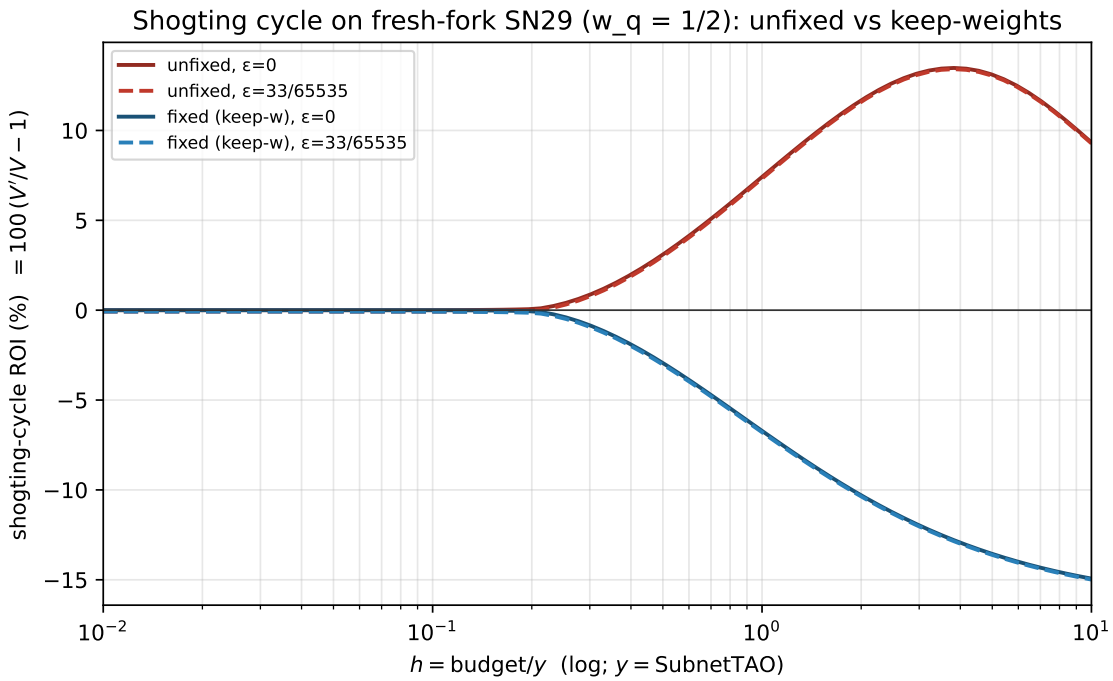


Figure 1: Shogting-cycle ROI (%) vs $h = \text{budget}/y$ on fresh-fork SN29 ($w_q = \frac{1}{2}$). Unfixed readd is profitable at large h; keep-weights stays non-positive. PNG twin: `no-arbitrage-shogting-roi.png`.

μ	This note	Relation
φ	φ (lift fraction)	Same object: $y_s/y = (1-\varphi)^{1/w_q}$ at $\varepsilon = 0$ is μ's $(1-\varphi)^{1/w_2}$ with $w_2 = w_q$.
w_2	w_q	Quote weight.
fees ignored	$\varepsilon = 0$ base case	Same restriction for the closed forms (8)–(9).
f_h	related to $h = V/y$	Size vs pool; our sizing (5) plus $S + B = V - C$ is the concrete single-wallet map.
$F < 1$ profitable	ROI > 0 profitable	Reciprocal conventions: $F_{\text{cyc}} := 1+\text{ROI} \approx 1/F$ when both describe the same fee-free short cycle *without* a weight ratchet.

Did we simplify, or does the logic differ?

- The lift identity (3) **reduces to** μ's $(1-\varphi)^{1/w_2}$ term — same algebra, independent derivation.
- The shogting-cycle ROI **differs** from μ's F once readd uses (4): that weight ratchet is the SN29 drain and is absent from the September fee-free F.
- Under keep-weights and $\varepsilon = 0$, the shogting cycle is the natural pallet realisation of μ's shorting round trip; our $\text{ROI}_{\text{fix}} \leq 0$ is the attacker-side statement matching $F \geq 1$ on that fee-free path.

7 Root fix

`readd_protocol_liquidity` credits $(\Delta y, \Delta x)$ **without** calling (4). Emissions still use `adjust_protocol_liquidit` Stake/unstake unchanged.

This is still weighted Balancer whenever $w_q \neq \frac{1}{2}$; the fresh fork happens to start at $\frac{1}{2}$. Keep-weights only skips asymmetric-join reweight on *lending* readd after skew.

Recommendation. Ship the keep-weights readd (the fixed shogting path) together with the k-neutrality burn of Section 8. Do not restore (4) on lending readd.

8 K-neutrality burn

Keep-weights alone stops the SN29 weight ratchet, but μ showed that a shogting cycle under fixed weights can only *increase* $\sqrt{k} = L = x^{w_b} y^{w_q}$ (proved for $w_q = \frac{1}{2}$): the cycle's loss stays in the pool and deepens it. Jacob (2026-10-30) requires lending cycles to leave k unchanged, so pools get thinner/more volatile while a lift is outstanding, not deeper after close.

Baseline. When the lift book goes idle→busy, snapshot pool reserves $(x^\star, y^\star)$ (storage `InvariantBaseline`). This defines

$$L^\star \;=\; (x^\star)^{w_b} (y^\star)^{w_q}$$

at the live weights (unchanged by readd).

Burn after last readd. On close/repay/forfeit/expiry, after a full weight-preserving credit of the lift legs, if the book is again idle:

$$\sigma \;=\; \Big(\frac{x^\star}{x'}\Big)^{w_b} \Big(\frac{y^\star}{y'}\Big)^{w_q},$$

where (x', y') are post-credit reserves. If $\sigma \geq 1$ ($L' \leq L^\star$) burn nothing. Otherwise keep σ of each reserve (spot fixed) and burn the rest. Floors at the pre-credit reserves ensure the burn never eats user funds or escrow — only surplus from the just-credited legs. Dissolution skips the burn (subnet winding down) and clears the baseline.

Any weights. The same σ works for arbitrary (w_b, w_q) with $w_b + w_q = 1$; equal weights reduce to $\sigma = L^\star / L'$. Burning TAO matches the existing burn-all-interest policy; surplus alpha is discarded from `SubnetAlphaIn`.

9 Pass/fail (keep-weights), search, slippage

Operational shipping criterion (unchanged intent): no self-contained sequence of length $\leq N = 8$ (multi-coldkey and trade-split variants included) yields attacker $\Delta > 0$ for $\varphi \leq \varphi_{\max}$ under keep-weights on live snapshots. Search PASS on SN29 / SN64 / SN51 / SN4 / SN119 (20 087 cells). Deliberate shallowness: $M(\varphi) \approx 1/(1-\varphi)$ while a lift is outstanding (Figure in prior revision; script `no_arbitrage_slippage_chart.py`).

A Contaminated clone runs (not the base case)

Sequential attacks on the lending clone, tip `4d3dc1585`:

run	y	printed w_1 ($= w_q$)	note
a13	7449.956	0.471657	started after a12 drain
a15	7152.475	0.450710	started after a13

Production-fee unfixed math still matches the reported free PnL (a13 +289.30 vs +289.27; a15 $\approx +122$) because those books really were skewed. They must not be used as the "mainnet w_q" base case — that is Section 3 ($w_q = \frac{1}{2}$).

References

[1] μ, September 2026 hand derivation of the fee-free short-cycle factor $F(f_h, \varphi, w_2)$. Cited for the mapping in Section 6.

[2] *Lift-based lending on main's Balancer pools*, `pallets/lending/docs/lift-lending-spec.md`.

[3] Lending-clone docs: Finney fork block #9,378,125; archive query of `SubnetTAO`/`SubnetAlphaIn`/`SwapBalancer` for SN29.

[4] Contaminated a13/a15 drain reports on clone tip `4d3dc1585` (Appendix A).